READING SHEET

# Metadata & SEO

Module 3 · Frontend: React & Next.js · Day 44 · Week 9

| PROGRAM | WORK TYPE | EST. TIME |
|---|---|---|
| **CodeOps · Full Stack** | **Revision + mini-project** | **2–3 hours** |

> **Tip How to use this sheet**
> A fast, secure application nobody can find is still a private project. This sheet covers the metadata API, social previews, structured data and what genuinely affects discoverability. Read Part 4 first if you are short of time — it explains why most of the SEO work was already done in weeks 8 and 9, and which of today's tags actually earn their place.

## Part 1 · The Metadata API

Every page needs a title and a description, and in the app router you **export** them rather than writing tags. There is no <head> in your JSX and no helper component to import.

app/menu/page.js — export an object

```
export const metadata = {
  title: "Menu",
  description: "Doro Wat, Tibs, Kitfo and Shiro, delivered
                across Addis Ababa.",
  alternates: { canonical: "/menu" }
};
```

A layout can export metadata too, applying to everything beneath it — which is how you set defaults once for a whole section.

### Titles that compose

app/layout.js — a template for every child

```
export const metadata = {
  title: {
    template: "%s · Addis Eats",
    default:  "Addis Eats — food delivery in Addis Ababa"
  }
};
// /menu -> "Menu · Addis Eats"    / -> the default
```

Each page supplies only its own part of the title; the site name is appended once, in one place. Keep titles short — search engines show roughly sixty characters, so put the distinguishing word first.

### Metadata from data

app/menu/[id]/page.js

```
export async function generateMetadata({ params }) {
  const dish = await getDish(params.id);
  if (!dish) return { title: "Dish not found" };
  return {
    title: dish.name,
    description: `${dish.name} — ${dish.price} ETB.` };
}
```

It runs on the server before the page renders and receives the same params. Calling getDish twice costs one request, because the framework dedupes it within a render.

> **Warning  Only in server components**
> A page marked "use client" cannot export metadata at all. If you need both, keep the page on the server and push the interactive part into a leaf — exactly the Day 38 pattern.

### What to set where

| SET IN | WHAT BELONGS THERE |
|---|---|
| Root layout | Title template, site name, metadataBase |
| Section layout | Defaults shared by that area |
| Static page | That page's own title and description |
| generateMetadata | Anything derived from the data |

> **Warning  metadataBase matters more than it looks**
> Set it in the root layout to your production URL. Without it, relative image paths in social previews resolve against nothing and every link you share arrives without its picture — the single most common reason a shared link looks broken on Telegram.

## Part 2 · Social Previews

When someone shares a dish in a Telegram group, what appears is decided by a handful of Open Graph tags. Four of them matter: **title**, **description**, **image** and **URL**.

```
                                                                the four that matter
export const metadata = {
  metadataBase: new URL("https://addiseats.et"),
  openGraph: {
    title: "Kitfo · Addis Eats",
    description: "Minced beef, mitmita, 320 ETB.",
    images: ["/og/kitfo.jpg"],   // 1200 × 630
    type: "website", locale: "en_ET" }
};
```

Use **1200 × 630** — anything else gets cropped unpredictably, usually through the middle of your text.

### Images by file convention

```
                                                          drop the file in, and it is used
app/opengraph-image.jpg        every page, unless overridden
app/icon.png                   the favicon
app/menu/opengraph-image.jpg   everything under /menu
```

The Day 36 idea again: reserved file names doing work by existing. A section overrides the site-wide preview simply by having its own file.

### Generating a preview per dish

app/menu/[id]/opengraph-image.jsx

```
import { ImageResponse } from "next/og";
export const size = { width: 1200, height: 630 };

export default async function Image({ params }) {
  const dish = await getDish(params.id);
  return new ImageResponse(
    <div style={{ fontSize: 72 }}>{dish.name}</div>);
}
```

It renders your markup to a picture on the server, so forty dishes get forty tailored cards without anyone opening a design tool. Note it supports only a limited subset of CSS — flexbox and basic properties, no grid — and failures are silent, so test it.

### Checking before you share

| SYMPTOM | USUAL CAUSE |
|---|---|
| No image on the card | metadataBase missing or wrong |
| The old image still shows | The platform cached it |
| Text cropped oddly | Not a 1200 × 630 image |
| Nothing but a blue link | No Open Graph tags at all |
| Works locally, not live | A localhost URL in metadataBase |

Load the deployed page, view source, and confirm the og tags are present with **absolute** URLs. Platforms cache previews for days, so change the image file name when you change the design or you will keep seeing the old one.

## Part 3 · Structured Data & Discovery

Metadata tells a crawler what a page is called. Structured data tells it what the page *is* — a restaurant, a menu item, a price in ETB.

JSON-LD for a dish

```
const jsonLd = {
  "@context": "https://schema.org",
  "@type": "MenuItem",
  name: dish.name,
  offers: { "@type": "Offer",
            price: dish.price, priceCurrency: "ETB" } };

<script type="application/ld+json"
  dangerouslySetInnerHTML={{ __html: JSON.stringify(jsonLd) }} />
```

> **Warning  It must match the page**
> What it buys you is a rich result rather than a plain link — a price, a rating, opening hours shown in the listing. But describing a price the page does not show is treated as deception and can cost you the rich result entirely. Generate it from the same data the page renders.

### sitemap and robots

app/sitemap.js

```
export default async function sitemap() {
  const dishes = await getDishes();
  return [
    { url: "https://addiseats.et/", priority: 1 },
    ...dishes.map((d) => ({
      url: `https://addiseats.et/menu/${d.id}` })) ];
}
```

Because it is a function, the sitemap is built from your real data at build time — a dish added next month appears without anyone remembering to update a file. Keep /checkout, /orders and anything behind sign-in out of it, and disallow them in robots.js.

### Canonical URLs

| THESE ARE ALL ONE PAGE | TO A CRAWLER |
|---|---|
| `/menu` | One page |
| `/menu?category=all` | Another page |
| `/menu?ref=telegram` | Another page |
| `/menu/` | Another page again |

Four URLs showing the same menu compete with each other instead of accumulating. `alternates: { canonical: "/menu" }` tells the crawler which one counts. You deliberately put the search term and page number in the query string on Day 41 — that was right for the person, and this is what it needs for the crawler.

## Part 4 · SEO in Practice

Most of what makes a page rank is not a tag. It is that the page loads quickly, says clearly what it is, and is built out of markup that means something.

| MATTERS A LOT | MATTERS LITTLE |
|---|---|
| Content that answers the query | Keyword density |
| Server-rendered HTML | The meta keywords tag |
| A fast LCP on a phone | Hidden text tricks |
| One h1 that describes the page | A perfect Lighthouse SEO score |
| Working links and clean URLs | Submitting to search engines |
| Alt text and semantic markup | Meta tag micro-tuning |

The rendering strategy from Day 37 decides what a crawler can read long before any tag does, and the LCP work from Day 43 affects ranking directly. Semantic HTML from Module 2 — header, nav, main, article and a sensible heading order — is still the structure a crawler reads.

### Writing a description worth showing

```
                                                    the same page, three descriptions
// ✗ "Welcome to our website. We are the best."
// ✗ "food delivery addis food delivery ethiopia food delivery"

// ✓ "Order Doro Wat, Tibs and Kitfo from kitchens in Bole and
//    Kazanchis. Delivery across Addis Ababa, paid with TeleBirr."
```

The description rarely affects ranking, but it is the sentence someone reads before deciding whether to click — write it for the person, in about 150 characters. The same description on forty dish pages tells a crawler they are duplicates, which is exactly what generateMetadata prevents.

### The mistakes to avoid

| MISTAKE | CONSEQUENCE |
|---|---|
| metadataBase not set | Every social preview loses its image |
| A client component page | No metadata export is possible |
| One description everywhere | Pages read as duplicates |
| Private routes in the sitemap | Crawl budget spent on redirects |
| Structured data that disagrees | The rich result is withdrawn |
| Several h1 elements | No clear subject for the page |

Every one of these is visible in the deployed page source in under a minute, with no tooling. Then verify with the platforms: Search Console for indexing, a social debugger for previews.

## Assigned Reading

**Required**

1. **nextjs.org — Metadata and OG images** What / why: the metadata object, generateMetadata, and the file conventions.
2. **Google — SEO starter guide** What / why: what genuinely affects ranking, from the people who decide it.

**Optional — go deeper**

- **The Open Graph protocol** — the tags themselves, from the specification that defines them.
- **Google — Structured data** — JSON-LD, the supported types, and the rich results each enables.

## Exercises

Verify each step by viewing source on the running page, not by reading your own code.

1. Set metadataBase and a title template in the root layout, with a default title.
2. Give every static route its own title and a description written for a person.
3. Add generateMetadata to the dish route using the name, price and summary.
4. Add a site-wide opengraph-image, then a generated one per dish with ImageResponse.
5. Add MenuItem JSON-LD generated from the same dish object the page renders.
6. Generate app/sitemap.js from your dishes, excluding the private routes.

7. Write robots.js disallowing /cart and /checkout and pointing at the sitemap.

## Mini-Project · Making Addis Eats Findable

> **Tip What you build**
> Every route with metadata worth showing, previews that render properly when a link is shared, structured data that matches the page, and a sitemap generated from your own records. The test is not the code — it is what appears in the page source and in a preview debugger.

### Requirements

- metadataBase set to the production URL, and a title template in the root layout.
- A distinct, human-readable description on every route.
- generateMetadata on the dish route, derived from the dish record.
- A site-wide opengraph-image plus a generated one per dish at 1200 × 630.
- MenuItem JSON-LD whose price and name match what the page displays.
- app/sitemap.js generated from real data, with private routes excluded and disallowed.
- A canonical URL on any route that accepts query parameters.

### Check yourself

- Does the page source show absolute URLs in every og:image tag?
- Do two different dish pages have two different descriptions?
- Does your sitemap contain any route that requires signing in?
- Does the JSON-LD price match the price on the page, in ETB?
- Is there exactly one h1 on each page, and does it describe the page?
- Would a link to a dish look worth opening if you saw it in a group chat?